

TBD

Author 1, *Member, IEEE* Author 2, *Senior Member, IEEE*...

Abstract—Abstract

Index Terms—Keywords

I. INTRODUCTION

Reference [1]

This paper is organized as follows. Section II... Section III...
Section IV... Section V...

II. BACKGROUND

III. MODEL

This section presents the simulation models for comparing proactive (BGP-like) and reactive (DNS-like) approaches for distributing Endpoint Identifier (EID) reachability information in Delay-Tolerant Networks (DTNs). Both protocols are implemented in OMNeT++ [2] and operate over identical physical topologies to ensure a fair comparison.

A. System Model

We consider a DTN consisting of N nodes interconnected by links with propagation delay d_{ij} between nodes i and j . Each node may host Bundle Protocol (BP) agents identified by unique EIDs [3]. The fundamental problem is: *how should nodes discover the Convergence-Layer Adapter (CLA) endpoint responsible for a given EID?*

Let $\mathcal{E} = \{e_1, e_2, \dots, e_M\}$ denote the set of EIDs in the network. For each EID e_k , there exists exactly one authoritative endpoint binding:

$$\text{endpoint}(e_k) = \langle \text{nodeId}, \text{claProtocol}, \text{port} \rangle \quad (1)$$

The goal is to enable any node i to resolve $\text{endpoint}(e_k)$ with:

- **Correctness:** The resolved endpoint matches the current authoritative binding
- **Timeliness:** Resolution completes within acceptable latency bounds
- **Efficiency:** Minimal network overhead and state distribution

Nodes in the network can assume different roles:

- **Publishers:** Nodes that originate EID bindings and announce them to the network
- **Clients:** Nodes that need to resolve EIDs to their endpoint bindings
- **Resolvers (DNS only):** Caching intermediaries that handle client queries
- **Authorities (DNS only):** Nodes that store authoritative EID records

B. Network Model

The primary experimental topology is a two-dimensional grid network:

$$G = (V, E) \text{ where } |V| = w \times h \quad (2)$$

Nodes are indexed $v_{i,j}$ for $i \in [0, w-1], j \in [0, h-1]$, with edges connecting adjacent nodes:

$$E = \{(v_{i,j}, v_{i',j'}) : |i - i'| + |j - j'| = 1\} \quad (3)$$

The network diameter (maximum shortest path) for a $w \times h$ grid is $D = w + h - 2$. For a 10×10 grid, $D = 18$ hops.

All links are modeled as delay channels with configurable propagation delay d . The default delay is $d = 10$ ms for terrestrial links, while deep-space scenarios use $d \in \{1, 5, 10, 20\}$ s. The channel model abstracts transmission time, focusing on propagation delay as appropriate for control-plane traffic with small message sizes.

For realistic scenarios, we model hierarchical topologies representing terrestrial disaster response (5–30 ms delays), lunar exploration networks (5 ms to 1.3 s delays), and Mars exploration networks (1 s to 720 s delays). These topologies include specialized channel types such as Earth-Moon links (1.3 s one-way light delay), Mars orbital relay channels (60 s), and interplanetary channels (720 s average).

C. BGP-like Push Model

The BGP-like model implements a path-vector routing protocol inspired by BGP-4 [4]. EID bindings are proactively flooded to all nodes upon publication, ensuring that each node maintains a local routing table with all known EIDs.

1) *Data Structures:* Each node maintains a routing table indexed by EID:

$$\text{bgpTable} : \mathbb{Z}^+ \rightarrow \text{BgpEntry} \quad (4)$$

where each entry contains:

- **eid:** The Endpoint Identifier
- **originId:** Publishing node ID
- **nextHop:** Gate index for forwarding
- **pathLength:** Distance in hops from the origin
- **version:** Update sequence number for churn handling
- **endpoint:** CLA binding information
- **path:** Full path vector for loop detection

2) *Message Types:* The protocol defines two message types:

- **BgpAnnounce** (64 + 4|path| bytes): Propagates EID reachability with path vector
- **BgpWithdraw** (32 bytes): Removes EID reachability

3) *Publication Algorithm*: When node p publishes EID e with endpoint ϵ :

- 1) Create local table entry with $\text{pathLength} = 0$ and $\text{path} = [p]$
- 2) Increment version: $v \leftarrow \text{publishedVersions}[e] + 1$
- 3) Construct announcement: $m \leftarrow \text{BgpAnnounce}(e, p, 0, v, \epsilon, t_{\text{now}}, [p])$
- 4) Flood to all neighbors: $\forall g \in \text{neighborGates} : \text{send}(m, g)$
- 4) *Announcement Processing*: Upon receiving announcement m at node n from gate g_{in} :

Step 1: Loop Detection. If $n \in m.\text{path}$, discard the message.

Step 2: Path Extension. Compute $\text{path}' \leftarrow m.\text{path} \cup \{n\}$ and $\text{pathLength}' \leftarrow m.\text{pathLength} + 1$.

Step 3: Acceptance Decision. Let $\text{existing} = \text{bgpTable}[m.\text{eid}]$. Accept if any condition holds:

$$\text{accept}(m) = \begin{cases} \text{true} & \text{if } \nexists \text{existing} \\ \text{true} & \text{if } m.\text{originId} = \text{existing}.\text{originId} \\ & \quad \wedge m.\text{version} > \text{existing}.\text{version} \\ \text{true} & \text{if } \text{pathLength}' < \text{existing}.\text{pathLength} \\ \text{true} & \text{if } \text{pathLength}' = \text{existing}.\text{pathLength} \\ & \quad \wedge m.\text{originId} < \text{existing}.\text{originId} \\ \text{false} & \text{otherwise} \end{cases} \quad (5)$$

Step 4: Table Update and Forwarding. If accepted, update the local table and forward to all neighbors except the arrival gate.

5) *Withdrawal Processing*: Withdrawal w for EID e is processed only if $w.\text{originId}$ matches the current entry's origin and $w.\text{version} \geq \text{existing}.\text{version}$. Upon acceptance, the entry is removed and the withdrawal is forwarded.

6) *Complexity Analysis: Convergence Time*: For a network with diameter D and link delay d :

$$T_{\text{conv}}^{\text{BGP}} = D \cdot d \quad (6)$$

For a 10×10 grid with $d = 10$ ms: $T_{\text{conv}}^{\text{BGP}} = 18 \times 0.01 = 0.18$ s.

State Complexity: Per-node state is $O(M)$ where M is the number of EIDs. Network-wide state is $O(N \cdot M)$ —full replication.

Message Complexity: $O(N)$ messages per EID publication in a grid topology.

D. DNS-like Pull Model

The DNS-like model implements an on-demand resolution protocol inspired by the Domain Name System [5]. EID bindings are stored at authoritative servers and resolved via queries with optional caching.

1) *Data Structures: Authority Record* (at authority nodes):

- **eid**: The Endpoint Identifier
- **publisherId**: Publisher node ID
- **endpoint**: CLA binding information
- **ttl**: Time-to-live in seconds
- **version**: Update sequence number
- **lastChangedTime**: Timestamp of last modification

Cache Entry (at resolver nodes):

- **eid**: The Endpoint Identifier
- **endpoint**: Cached CLA binding
- **ttl**: Original TTL value
- **expiryTime**: When the cache entry expires
- **recordTime**: Authority's lastChangedTime (for staleness detection)

2) *Message Types*:

- **DnsQuery** (48 bytes): Client query for an EID
- **DnsResponse** (80 bytes): Response with endpoint binding
- **DnsRegister** (72 bytes): Publisher registration at authority
- **DnsDeregister** (24 bytes): Publisher withdrawal

3) *Registration*: When publisher p registers EID e at authority a :

If $p = a$ (local authority), the record is stored directly. Otherwise, a **DnsRegister** message is sent to the authority. Authority assignment uses modular hashing:

$$a = (e \bmod |\mathcal{A}|) + a_{\text{first}} \quad (7)$$

where $\mathcal{A}$ is the set of authority nodes.

4) *Query Resolution*: Client-initiated query for EID e :

- 1) **Local Cache Check**: If $e \in \text{dnsCache}$ and not expired, return cache hit
- 2) **Cache Miss**: Create query and forward to resolver
- 3) **Resolver Processing**: Check resolver cache; if miss, forward to authority
- 4) **Authority Processing**: Look up record and return response
- 5) **Response Path**: Response traverses back, with resolvers caching the result

5) *Caching Mechanism*: Cache entries are inserted with expiry time $t_{\text{now}} + \tau$ where τ is the TTL. When the cache exceeds capacity, expired entries are removed first, then the entry with earliest expiry time is evicted.

The cache hit rate is:

$$\text{hitRate} = \frac{\sum \text{cacheHits}}{\sum \text{cacheHits} + \sum \text{cacheMisses}} \quad (8)$$

6) *Complexity Analysis: Query Latency (cache miss)*:

$$L_{\text{query}} \approx 2 \cdot D \cdot d \quad (9)$$

State Complexity: Authority state is $O(M)$. Resolver cache is $O(\min(M, C))$ where C is cache capacity. Network-wide state is $O(M)$ —centralized.

Message Complexity: $O(1)$ messages per registration; $O(1)$ per query.

E. Fair Comparison Design

To ensure a fair comparison, both protocols operate over the same physical grid topology. However, DNS in practice runs over an already-routed IP network, whereas BGP-like flooding operates at the network layer. To account for this, we implement a “direct mode” for DNS.

1) *Direct Mode*: In direct mode (`dnsDirectMode=true`), DNS queries use OMNeT++'s `sendDirect()` mechanism with RTT-based delays:

$$\text{RTT}_{\text{DNS}} = 2 \cdot D \cdot d \quad (10)$$

This models DNS running over a pre-routed IP underlay, isolating the protocol comparison from routing convergence effects. The approach ensures that:

- BGP announcements propagate through the physical topology hop-by-hop
- DNS queries logically traverse: Client $\rightarrow$ Resolver $\rightarrow$ Authority $\rightarrow$ Resolver $\rightarrow$ Client
- Both protocols experience comparable network latencies

F. Staleness and Accuracy Model

A central `GroundTruth` module maintains authoritative records for all EIDs, enabling correctness verification.

1) *BGP Accuracy*: A node's BGP entry for EID e is *correct* if:

$$\text{correct}_{\text{BGP}}(n, e) = \begin{cases} \text{bgpTable}[e].\text{endpoint} = \text{GT}[e].\text{endpoint} & \text{if active} \\ e \notin \text{bgpTable} & \text{if withdrawn} \end{cases} \quad (11)$$

Network-wide BGP accuracy:

$$\text{Accuracy}_{\text{BGP}}(e) = \frac{|\{n : \text{correct}_{\text{BGP}}(n, e)\}|}{N} \quad (12)$$

2) *DNS Staleness*: A DNS response is *stale* if the cached record predates the last authoritative change:

$$\text{stale}(r) = r.\text{recordTime} < \text{GT}[r.\text{eid}].\text{lastChangeTime} \quad (13)$$

Under churn with interval Δ_{churn} :

$$P(\text{stale}) \approx \frac{\Delta_{\text{churn}}}{\tau} \text{ for } \tau > \Delta_{\text{churn}} \quad (14)$$

where τ is the TTL.

3) *Convergence Detection*: Convergence for EID e occurs when $\geq 99\%$ of nodes have correct information:

$$\text{converged}(e, t) = \frac{|\{n : \text{correct}(n, e, t)\}|}{N} \geq 0.99 \quad (15)$$

We distinguish *initial convergence* (first publication) from *churn convergence* (subsequent updates).

G. Query Distribution Model

Query patterns follow Zipf's law [6]:

$$P(\text{query for EID } e_k) \propto \frac{1}{k^\alpha} \quad (16)$$

where k is the rank (1 = most popular) and α is the skewness parameter. Values of $\alpha = 0$ yield uniform distribution, while $\alpha = 1.0$ approximates typical web traffic patterns.

H. Metrics Summary

Table I summarizes the key metrics collected during simulation.

TABLE I
SIMULATION METRICS

Category	Metric	Unit
Overhead	Total bytes sent	bytes
	Total messages	count
Latency	Discovery latency	seconds
	Convergence time	seconds
	Query latency	seconds
Accuracy	BGP accuracy	[0, 1]
	DNS accuracy	[0, 1]
	Stale rate	[0, 1]
State	BGP table size	entries
	DNS cache size	entries

TABLE II
PROTOCOL COMPLEXITY COMPARISON

Aspect	BGP (Push)	DNS (Pull)
State per node	$O(M)$	$O(1)$ client, $O(C)$ resolver
Network state	$O(N \cdot M)$	$O(M)$
Publication cost	$O(N)$ messages	$O(1)$ messages
Query cost	$O(1)$ (local)	$O(D)$ hops
Churn cost	$O(N)$ per change	$O(1)$ per change
Convergence time	$O(D \cdot d)$	N/A (on-demand)
Discovery latency	$D \cdot d$	$2 \cdot D \cdot d$
Resolution latency	≈ 0 (local)	$2 \cdot D \cdot d$

I. Complexity Comparison

Table II compares the theoretical complexity of both approaches.

IV. EVALUATION

V. CONCLUSIONS

REFERENCES

- [1] A. Alhilal, T. Braud, and P. Hui, "The sky is not the limit anymore: Future architecture of the interplanetary internet," *IEEE Aerospace and Electronic Systems Magazine*, vol. 34, no. 8, pp. 22–32, 2019.
- [2] A. Varga, "OMNeT++," in *Modeling and Tools for Network Simulation*. Springer, 2010, pp. 35–59.
- [3] S. Burleigh, K. Fall, and E. Birrane, "Bundle Protocol Version 7," RFC 9171, January 2022.
- [4] Y. Rekhter, T. Li, and S. Hares, "A Border Gateway Protocol 4 (BGP-4)," RFC 4271, January 2006.
- [5] P. Mockapetris, "Domain Names - Implementation and Specification," RFC 1035, November 1987.
- [6] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, "Web caching and zipf-like distributions: Evidence and implications," in *IEEE INFOCOM'99. Conference on Computer Communications. Proceedings. Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies*, vol. 1. IEEE, 1999, pp. 126–134.